```
(-split-at position list);; Syntax

(-split-at 2 '(1 2 3 4 5))
((1 2) (3 4 5))
```

The '-split-with' function takes a predicate function and an input list. It applies the predicate function to the beginning of the input list, and takes the elements until they match the predicate. The remaining part of the input list is simply returned as the second list. A couple of examples are shown below:

```
(-split-with predicate list);; Syntax

(-split-with 'even? '(4 5 6 7))
((4) (5 6 7))

(-split-with 'even? '(1 2 3 4))
(nil (1 2 3 4))
```

The '-split-when' function splits the input list wherever the element in the input list matches the predicate function. For example:

```
(-split-when function list);; Syntax

(-split-when 'even? '(1 2 3 4 5))
((1) (3) (5))
```

The '-separate' function takes a predicate function and an input list, and creates two lists in a single pass. All the elements matching the predicate function are in the first list, and the rest are present in the second list. An example is shown below:

```
(-separate predicate list);; Syntax

(-separate 'even? '(1 2 3 4 5))
((2 4) (1 3 5))
```

If you want to split a list into buckets with a specific size, you can use the '-partition' function. If there are insufficient elements in the list, those specific elements are not returned. For example:

```
(-partition size list);; Syntax

(-partition 3 '(1 2 3 4 5))
((1 2 3))
```

The '-partition-all' function is similar to the above '-partition' function, except that if the last group contains fewer items, they are still retained. An example is given below:

```
(-partition-all size list);; Syntax

(-partition-all 3 '(1 2 3 4 5))
((1 2 3) (4 5))
```

The '-partition-in-steps' function takes three arguments, - *a* position, offset and an input list. It splits the input list into sizes, but, at the specified offset. In the following example, the input list '(1 2 3 4 5)' is divided into buckets of size two, but, at every one element offset.

```
(-partition-in-steps size offset list);; Syntax

(-partition-in-steps 2 1 '(1 2 3 4 5))
((1 2) (2 3) (3 4) (4 5))
```

The '-partition-all-in-steps' function is similar to the '-partition-in-steps' function, except that it retains the last group, even if it has fewer elements than the bucket size. For example:

```
(-partition-all-in-steps size offset list);; Syntax

(-partition-all-in-steps 2 1 '(1 2 3 4 5))
((1 2) (2 3) (3 4) (4 5) (5))
```

The '-partition-by' function takes a predicate function and an input list, and splits the input list as and when the predicate function returns a different value. An example is given below for reference:

```
(-partition-by function list);; Syntax

(-partition-by 'even? '(1 2 2 3 4 5 5))
((1) (2 2) (3) (4) (5 5))
```

The 'group-by' function separates the input list into two lists, in which the keys are the function applied to each element in the list. For example:

```
(-group-by function list);; Syntax

(-group-by 'even? '(1 1 2 3 4 5 5 6 8))
((nil 1 1 3 5 5) (t 2 4 6 8))
```

## Indexing

The '-elem-index' function takes an element and an input list, and returns the index in the list where there is a first match for the element. An example is given below:

```
(-elem-index element list) ;; Syntax
```